UNIVERSIDAD COOPERATIVA DE COLOMBIA

Facultad de Ingeniería – Programa de Ingeniería de Sistemas

Optativa 1 – Desarrollo de Software Web

LABORATORIO 06

Evolución de una aplicación HelpDesk mediante API, Fetch y Async/Await

Duración: 3 horas

Tecnologías: HTML5 · CSS3 · JavaScript ES6+ · Fetch API · JSON · Async/Await · Visual Studio Code · Live Server

Modalidad: trabajo práctico aplicado sobre la aplicación desarrollada en el Laboratorio 05

1. Enunciado general

La empresa ya cuenta con una aplicación HelpDesk desarrollada en los laboratorios anteriores. En el Laboratorio 05 se construyó una interfaz capaz de crear, editar, eliminar, buscar y filtrar tickets. El siguiente paso consiste en conectar esta aplicación con una fuente externa de datos.

El estudiante deberá tomar la aplicación existente y convertirla en una aplicación web que consulte y envíe información mediante una API. No se debe iniciar un proyecto diferente. Todas las nuevas funcionalidades deben integrarse al HelpDesk existente.

El propósito es pasar de una aplicación que trabaja solamente con datos locales en JavaScript a una arquitectura en la que la interfaz se comunica con un servicio externo mediante solicitudes HTTP y datos JSON.

2. Situación problema

La empresa necesita que los tickets de soporte puedan ser consultados desde un servicio externo. La aplicación debe mostrar un indicador mientras obtiene la información, informar al usuario cuando exista un error, mantener la búsqueda y los filtros, permitir registrar nuevos tickets y preparar las operaciones de actualización y eliminación.

El resultado esperado es una aplicación empresarial funcional en navegador, desarrollada en Visual Studio Code, que evidencie el flujo:

API → fetch() → JSON → JavaScript → filtros/procesamiento → DOM → interfaz

3. Objetivos de aprendizaje

- Consumir información desde una API mediante fetch().
- Aplicar async/await para controlar operaciones asíncronas.
- Interpretar respuestas JSON.
- Transformar datos externos al modelo de la aplicación.
- Integrar datos externos con el DOM.
- Mantener búsqueda, filtros y renderizado del HelpDesk.
- Implementar solicitudes GET y POST.
- Aplicar DELETE como reto de profundización.
- Gestionar estados de carga y errores.
- Comprender la transición hacia un backend y una base de datos.

4. Producto final esperado

- Aplicación HelpDesk evolucionada a partir del Laboratorio 05.
- Carpeta organizada con index.html, css/styles.css, js/api.js y js/app.js.
- Consulta de tickets desde una API.
- Tabla dinámica alimentada por datos recibidos.
- Búsqueda y filtros funcionando.
- Formulario para crear tickets.
- Mensaje de carga y mensaje de error.
- Eliminación mediante API como reto.
- Interfaz empresarial funcional.

5. Reglas de trabajo

1. No borrar la aplicación del Laboratorio 05.
2. Conservar los nombres y la lógica que puedan reutilizarse.
3. Separar las funciones de comunicación con la API en api.js.
4. Dejar la manipulación de la interfaz y los eventos principalmente en app.js.
5. Probar cada funcionalidad antes de avanzar.
6. Tomar una captura de cada etapa solicitada.
7. Comentar el código que represente una decisión importante.

6. Estructura del proyecto

Crear o reorganizar el proyecto con la siguiente estructura:

```
lab06-helpdesk-api/
├── css/
│   └── styles.css
├── js/
│   ├── api.js
│   └── app.js
└── index.html
```

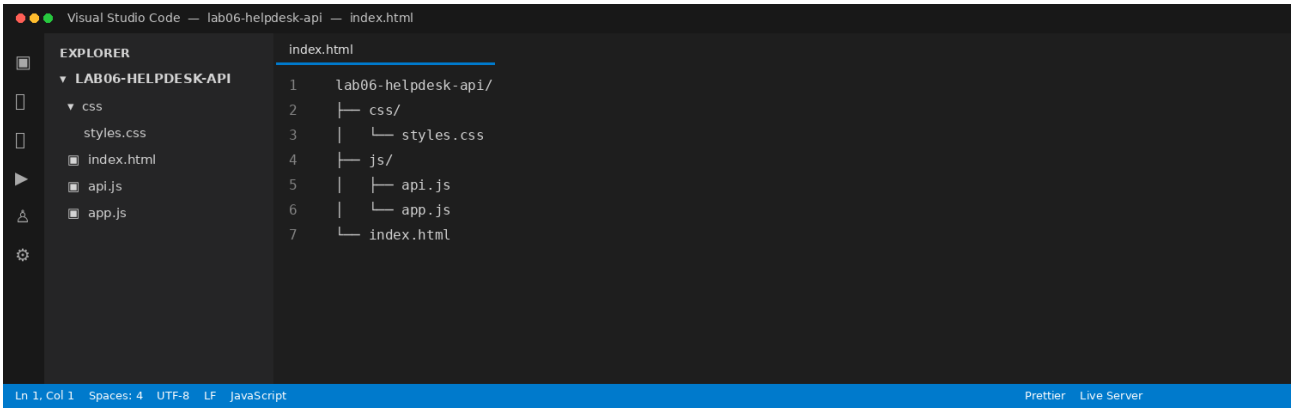


Figura 1. Estructura del proyecto en una interfaz tipo Visual Studio Code.

7. Paso 1 — Construir la interfaz base

El estudiante debe conservar la estructura de HelpDesk del laboratorio anterior y adaptarla para mostrar un encabezado, formulario de tickets, búsqueda, filtros y tabla.

El siguiente código corresponde al primer bloque de index.html.

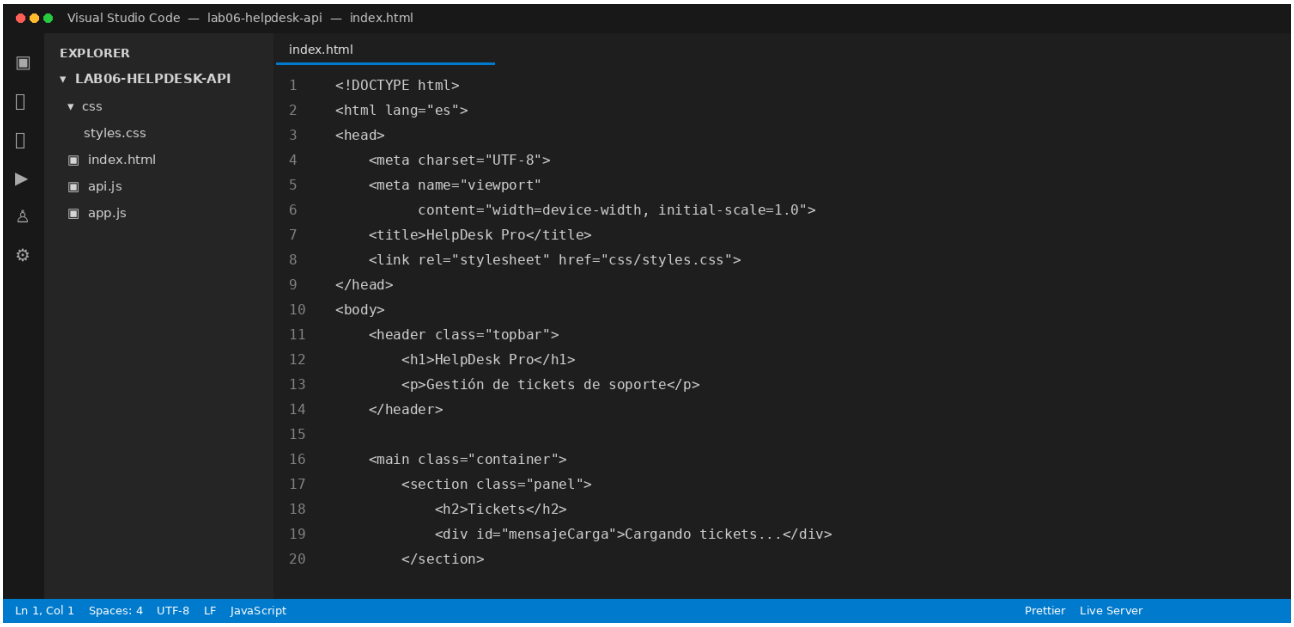


Figura 2. Pantallazo del código de index.html en un entorno visual tipo Visual Studio Code.

Código utilizado — index.html (parte 1)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
  <title>HelpDesk Pro</title>
```



```
<link rel="stylesheet" href="css/styles.css">

</head>
<body>
  <header class="topbar">
    <h1>HelpDesk Pro</h1>
    <p>Gestión de tickets de soporte</p>
  </header>

  <main class="container">
    <section class="panel">
      <h2>Tickets</h2>
      <div id="mensajeCarga">Cargando tickets...</div>
    </section>
```

8. Paso 2 — Formulario y filtros

Agregar los campos necesarios para registrar un ticket y conservar los filtros que ya tenía la aplicación. Los filtros deben continuar trabajando sobre el array tickets.

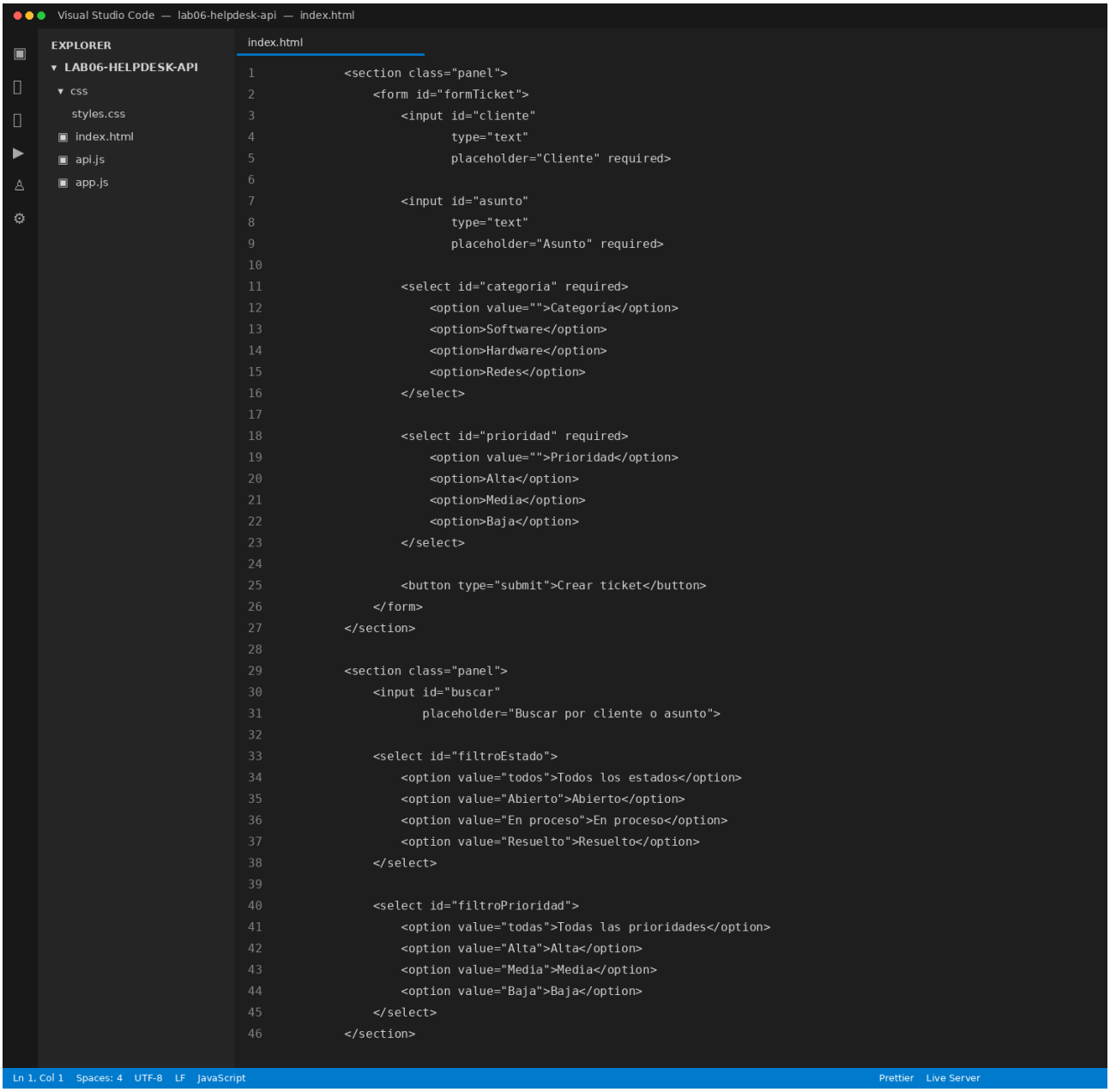


Figura 3. Pantallazo del formulario y filtros en Visual Studio Code.

Código utilizado — index.html (parte 2)

```
    <section class="panel">
      <form id="formTicket">
        <input id="cliente"
               type="text"
               placeholder="Cliente" required>

        <input id="asunto"
               type="text"
               placeholder="Asunto" required>




        <select id="categoria" required>
          <option value="">Categoría</option>
          <option>Software</option>
          <option>Hardware</option>
          <option>Redes</option>
        </select>

        <select id="prioridad" required>
          <option value="">Prioridad</option>
          <option>Alta</option>
          <option>Media</option>
          <option>Baja</option>
        </select>

        <button type="submit">Crear ticket</button>
      </form>
    </section>

    <section class="panel">
      <input id="buscar"
             placeholder="Buscar por cliente o asunto">

      <select id="filtroEstado">
        <option value="todos">Todos los estados</option>
        <option value="Abierto">Abierto</option>
        <option value="En proceso">En proceso</option>
        <option value="Resuelto">Resuelto</option>
      </select>

      <select id="filtroPrioridad">
        <option value="todas">Todas las prioridades</option>
        <option value="Alta">Alta</option>
        <option value="Media">Media</option>
        <option value="Baja">Baja</option>
      </select>
    </section>
```

9. Paso 3 — Tabla y conexión de scripts

La tabla será llenada dinámicamente mediante JavaScript. Los archivos api.js y app.js se cargan al final del documento para que los elementos HTML estén disponibles cuando se ejecuten las funciones.

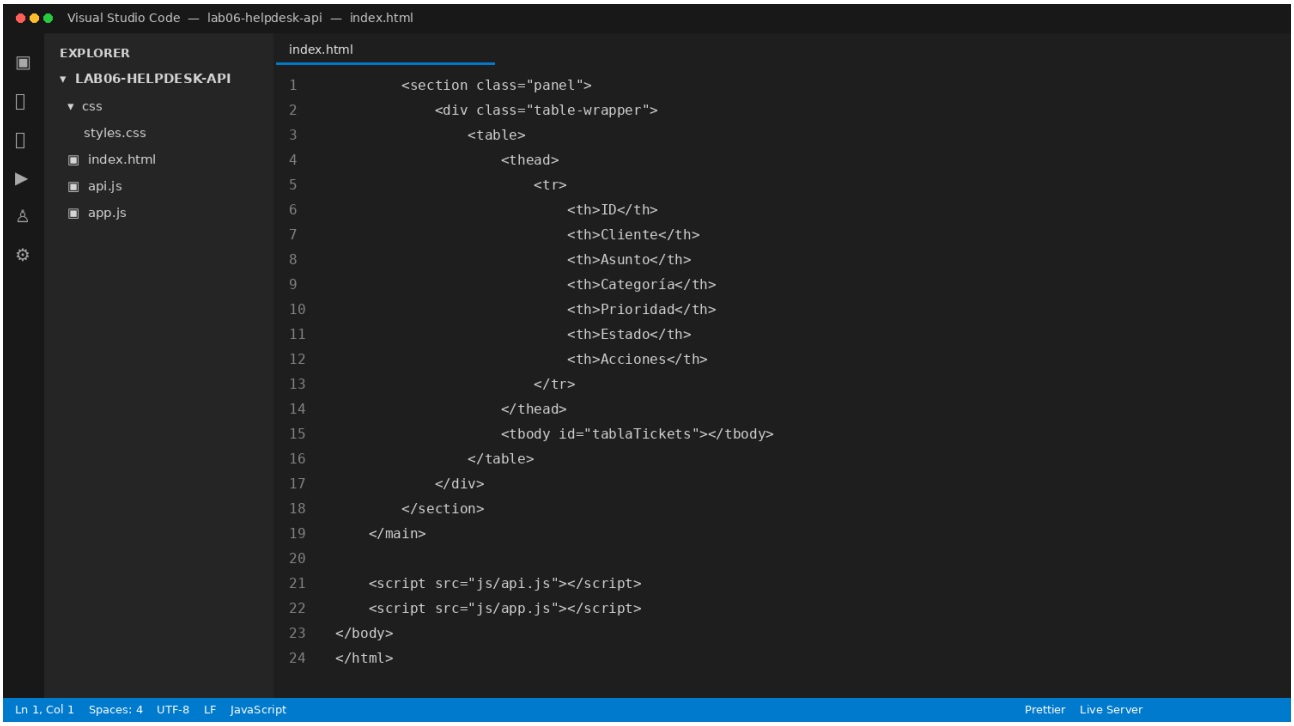


Figura 4. Pantallazo de la tabla dinámica y conexión de scripts.

Código utilizado — index.html (parte 3)

```
<section class="panel">
  <div class="table-wrapper">
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Cliente</th>
          <th>Asunto</th>
          <th>Categoría</th>
          <th>Prioridad</th>
          <th>Estado</th>
          <th>Acciones</th>
        </tr>
      </thead>
      <tbody id="tablaTickets"></tbody>
    </table>
  </div>
</section>
</main>

<script src="js/api.js"></script>
<script src="js/app.js"></script>
```

</body>
</html>

10. Paso 4 — Crear api.js

En este paso se separa la comunicación con el servidor de la lógica de interfaz. Se utiliza JSONPlaceholder como API de práctica. Importante: es una API educativa de demostración; las operaciones de escritura no constituyen una base de datos propia permanente.

La URL de práctica utilizada es:

https://jsonplaceholder.typicode.com/posts

El archivo api.js tendrá funciones para consultar, crear y eliminar.

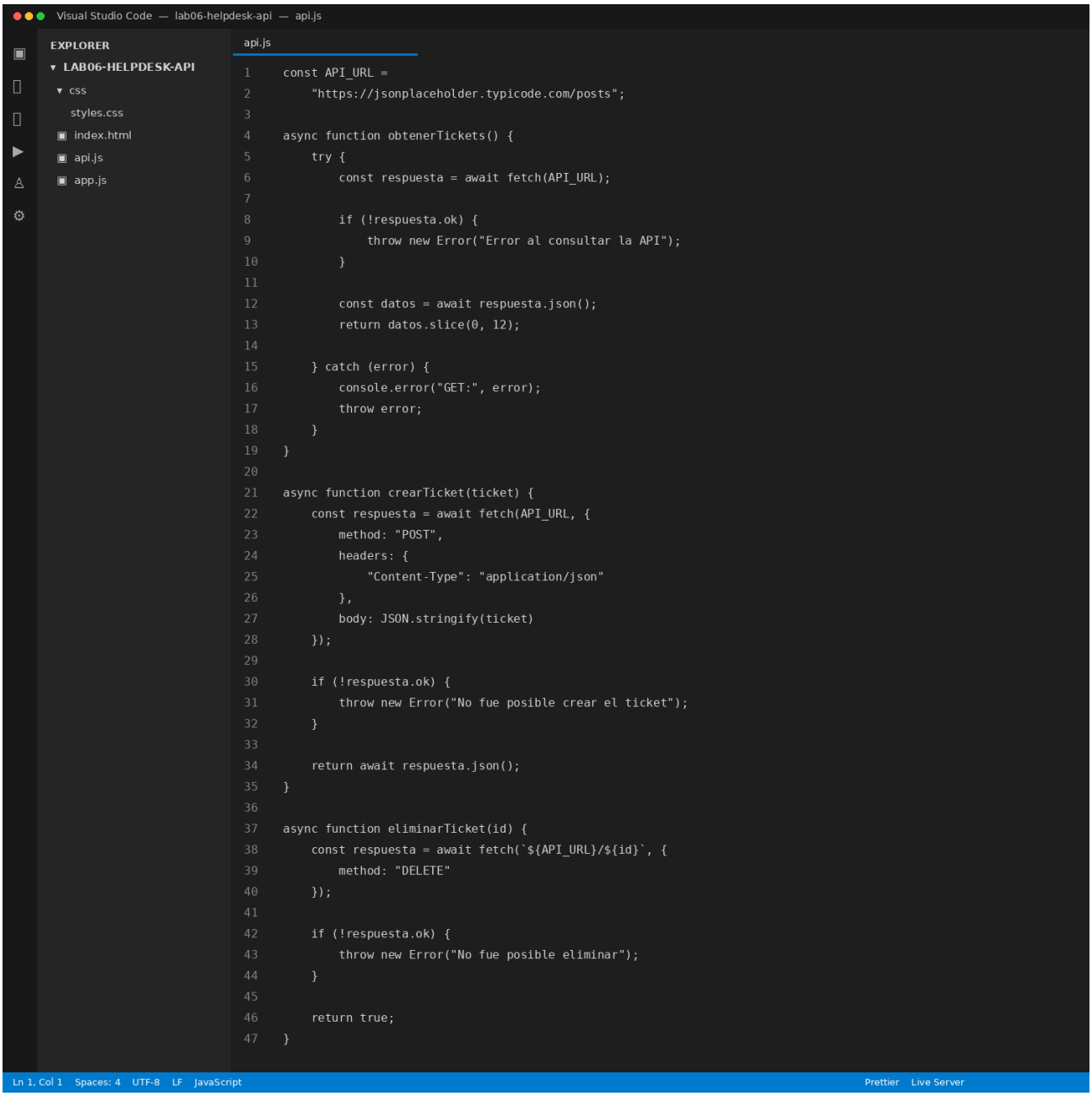


Figura 5. Pantallazo realista del código api.js en Visual Studio Code.

Código utilizado — api.js

```
const API_URL =
   "https://jsonplaceholder.typicode.com/posts";

async function obtenerTickets() {
   try {
      const respuesta = await fetch(API_URL);

      if (!respuesta.ok) {
         throw new Error("Error al consultar la API");
      }

      const datos = await respuesta.json();
      return datos.slice(0, 12);

   } catch (error) {
      console.error("GET:", error);
      throw error;
   }
}

async function crearTicket(ticket) {
   const respuesta = await fetch(API_URL, {
      method: "POST",
      headers: {
         "Content-Type": "application/json"
      },
      body: JSON.stringify(ticket)
   });

   if (!respuesta.ok) {
      throw new Error("No fue posible crear el ticket");
   }

   return await respuesta.json();
}

async function eliminarTicket(id) {
   const respuesta = await fetch(`${API_URL}/${id}`, {
      method: "DELETE"
   });


  if (!respuesta.ok) {
     throw new Error("No fue posible eliminar");
  }

  return true;
}
```

11. Paso 5 — Modelo de datos de la aplicación

La API de práctica devuelve objetos que no tienen exactamente los mismos campos que necesita el HelpDesk. Por eso se crea una función transformarTicket(). Esta función convierte el objeto externo al modelo utilizado por la aplicación.

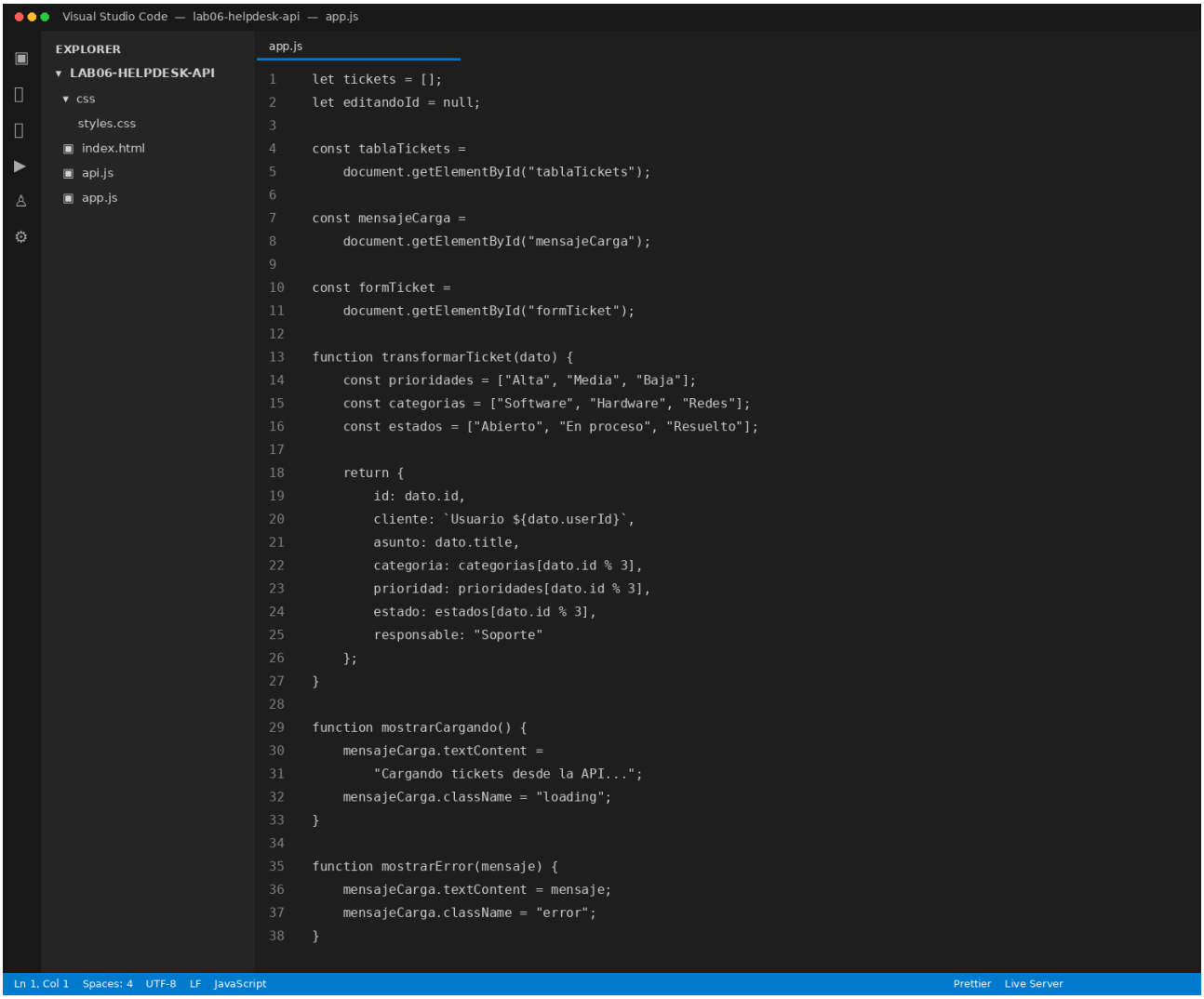


Figura 6. Pantallazo del modelo de datos y funciones iniciales en app.js.

▲ **Código utilizado — app.js (parte 1)**

```
let tickets = [];
let editandoId = null;

const tablaTickets =
  document.getElementById("tablaTickets");

const mensajeCarga =
  document.getElementById("mensajeCarga");

const formTicket =
document.getElementById("formTicket");
```

```
function transformarTicket(dato) {
    const prioridades = ["Alta", "Media", "Baja"];
    const categorias = ["Software", "Hardware", "Redes"];
    const estados = ["Abierto", "En proceso", "Resuelto"];

    return {
        id: dato.id,
        cliente: `Usuario ${dato.userId}`,
        asunto: dato.title,
        categoria: categorias[dato.id % 3],
        prioridad: prioridades[dato.id % 3],
        estado: estados[dato.id % 3],
        responsable: "Soporte"
    };
}

function mostrarCargando() {
    mensajeCarga.textContent =
        "Cargando tickets desde la API...";
    mensajeCarga.className = "loading";
}

function mostrarError(mensaje) {
    mensajeCarga.textContent = mensaje;
    mensajeCarga.className = "error";
}
```

12. Paso 6 — Renderizado dinámico y filtros

La información recibida desde la API debe aparecer en la misma tabla del HelpDesk. El estudiante debe conservar el concepto de renderizado dinámico y reutilizar la búsqueda y los filtros del laboratorio anterior.



Figura 7. Pantallazo del renderizado y filtros.

Código utilizado — app.js (parte 2)

```
function renderTickets() {
    const visibles = obtenerTicketsVisibles();
    tablaTickets.innerHTML = "";

    if (visibles.length === 0) {
        tablaTickets.innerHTML = `
            <tr>
                <td colspan="7">
                    No hay tickets para mostrar
                </td>
            </tr>`;
        return;
    }

    visibles.forEach(ticket => {
        const fila = document.createElement("tr");

        fila.innerHTML = `
            <td>${ticket.id}</td>
            <td>${ticket.cliente}</td>
            <td>${ticket.asunto}</td>
            <td>${ticket.categoria}</td>
            <td>${ticket.prioridad}</td>
            <td>${ticket.estado}</td>
            <td>
                <button data-id="${ticket.id}"
                        data-action="eliminar">
                    Eliminar
                </button>
            </td>`;

        tablaTickets.appendChild(fila);
    });
}

function obtenerTicketsVisibles() {
    const texto =
        document.getElementById("buscar")
        .value.trim().toLowerCase();

    const estado =
        document.getElementById("filtroEstado").value;


     const prioridad =
         document.getElementById("filtroPrioridad").value;

     return tickets.filter(ticket => {
         const coincideTexto =
             ticket.cliente.toLowerCase().includes(texto) ||
             ticket.asunto.toLowerCase().includes(texto);
```

```
        const coincideEstado =
            estado === "todos" ||
            ticket.estado === estado;

        const coincidePrioridad =
            prioridad === "todas" ||
            ticket.prioridad === prioridad;

        return coincideTexto &&
               coincideEstado &&
               coincidePrioridad;
    });
}
```

13. Paso 7 — GET, Async/Await y POST

Ahora la aplicación deja de depender exclusivamente del array escrito manualmente. cargarTickets() obtiene los datos desde la API y posteriormente los transforma y renderiza.

El formulario también utiliza una solicitud POST para enviar un nuevo ticket.



Figura 8. Pantallazo de carga desde API, POST y eventos del formulario.

Código utilizado — app.js (parte 3)

```
async function cargarTickets() {
  try {
    mostrarCargando();

    const datos = await obtenerTickets();

    tickets = datos.map(transformarTicket);

    mensajeCarga.textContent =
      `✓ ${tickets.length} tickets cargados`;
    mensajeCarga.className = "success";

    renderTickets();

  } catch (error) {
    mostrarError(
      "No fue posible conectar con el servidor."
    );
  }
}

formTicket.addEventListener("submit",
  async (event) => {

    event.preventDefault();

    const ticket = {
      title:
        document.getElementById("asunto").value,
      cliente:
        document.getElementById("cliente").value,
      categoria:
        document.getElementById("categoria").value,
      prioridad:
        document.getElementById("prioridad").value,
      estado: "Abierto",
      responsable: "Soporte"
    };

    try {
      const respuesta =
        await crearTicket(ticket);


        tickets.unshift({
          ...ticket,
          id: respuesta.id
        });

        formTicket.reset();
        renderTickets();
```

```
    } catch (error) {
        mostrarError("Error al crear el ticket.");
    }
});

document.getElementById("buscar")
    .addEventListener("input", renderTickets);

document.getElementById("filtroEstado")
    .addEventListener("change", renderTickets);

document.getElementById("filtroPrioridad")
    .addEventListener("change", renderTickets);

cargarTickets();
```

14. Paso 8 — Delegación de eventos y DELETE

La aplicación debe reutilizar la delegación de eventos trabajada en el Laboratorio 05. El botón Eliminar se identifica mediante data-id y data-action. Después se llama a la función eliminarTicket() de api.js.

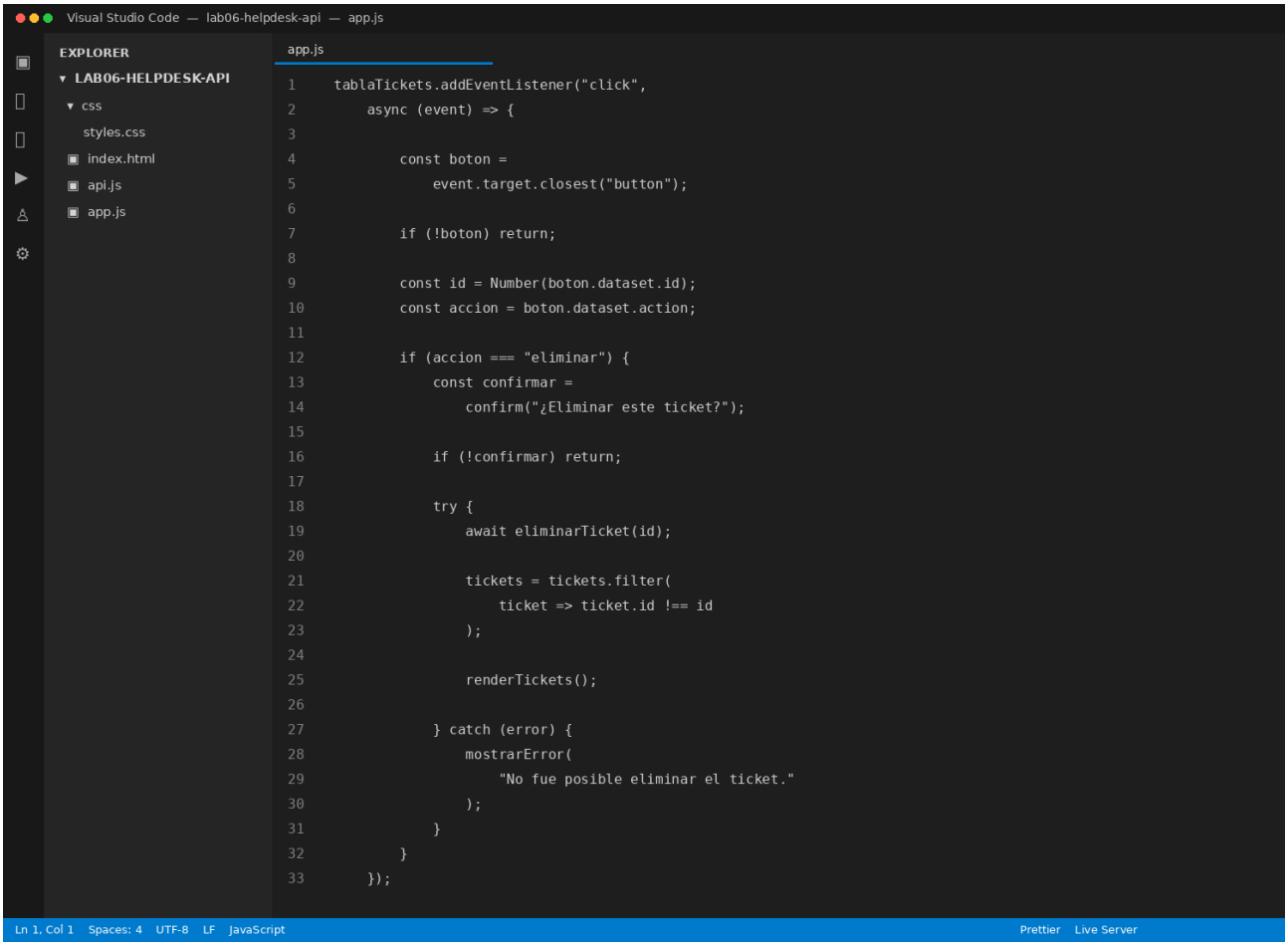


Figura 9. Pantallazo de delegación de eventos y eliminación.

Código utilizado — app.js (parte 4)

```
tablaTickets.addEventListener("click",
    async (event) => {

        const boton =
            event.target.closest("button");

        if (!boton) return;

        const id = Number(boton.dataset.id);
        const accion = boton.dataset.action;

        if (accion === "eliminar") {
            const confirmar =
                confirm("¿Eliminar este ticket?");

            if (!confirmar) return;

            try {
                await eliminarTicket(id);

                tickets = tickets.filter(
                    ticket => ticket.id !== id
                );

                renderTickets();

            } catch (error) {
                mostrarError(
                    "No fue posible eliminar el ticket."
                );
            }
        }
    });
```

15. Paso 9 — Estilos de la aplicación

La interfaz debe conservar una presentación empresarial: encabezado, paneles, formulario, filtros y tabla.

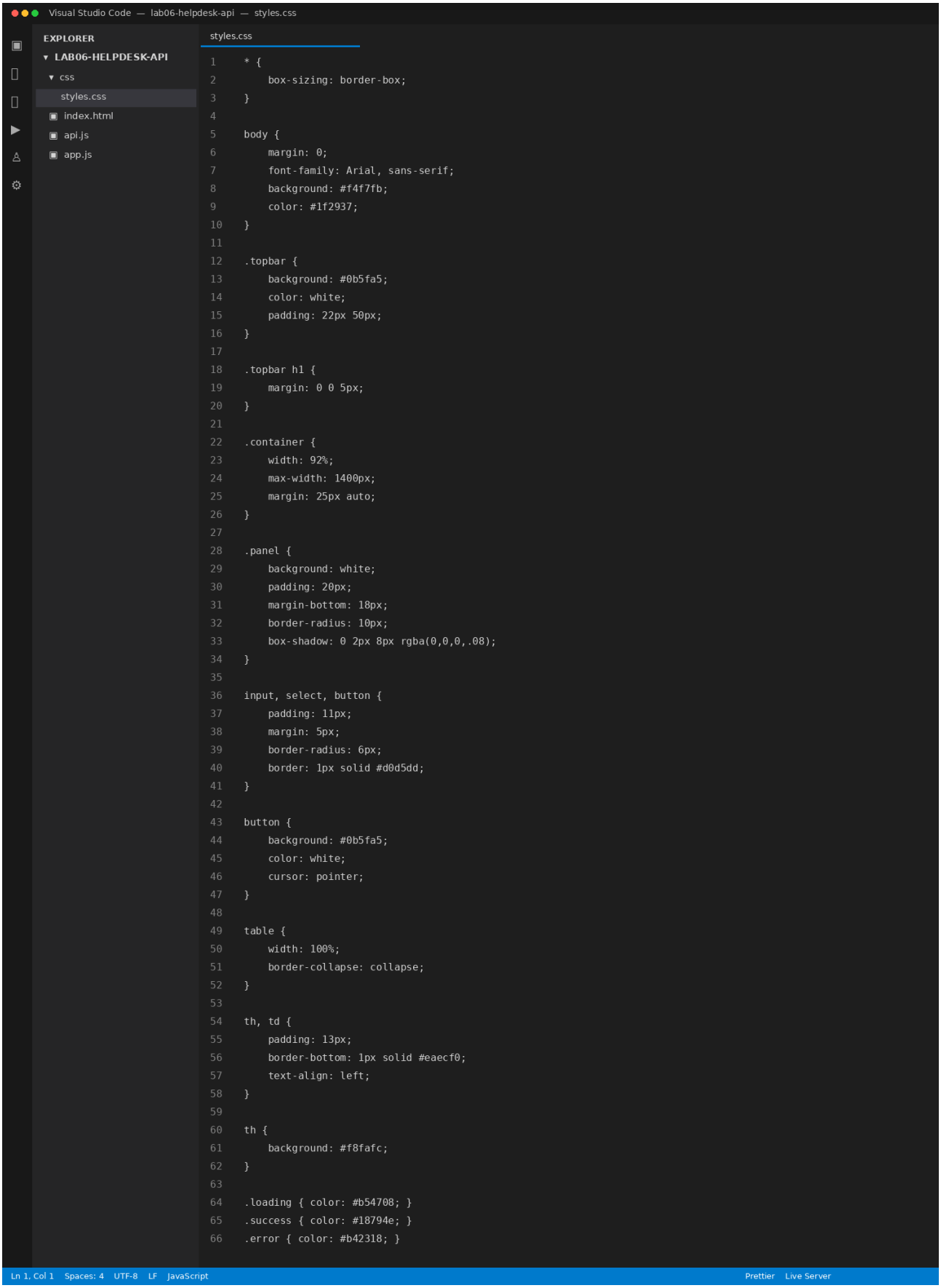


Figura 10. Pantallazo del archivo styles.css.

Código utilizado — styles.css

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f4f7fb;
    color: #1f2937;
}

.topbar {
    background: #0b5fa5;
    color: white;
    padding: 22px 50px;
}

.topbar h1 {
    margin: 0 0 5px;
}

.container {
    width: 92%;
    max-width: 1400px;
    margin: 25px auto;
}

.panel {
    background: white;
    padding: 20px;
    margin-bottom: 18px;
    border-radius: 10px;
    box-shadow: 0 2px 8px rgba(0,0,0,.08);
}

input, select, button {
    padding: 11px;
    margin: 5px;
    border-radius: 6px;
    border: 1px solid #d0d5dd;
}

button {
    background: #0b5fa5;
    color: white;
    cursor: pointer;
}

table {
    width: 100%;
    border-collapse: collapse;
}
```

```
th, td {
    padding: 13px;
    border-bottom: 1px solid #eaecf0;
    text-align: left;
}

th {
    background: #f8fafc;
}

.loading { color: #b54708; }
.success { color: #18794e; }
.error { color: #b42318; }
```

16. Paso 10 — Resultado en el navegador

Al ejecutar Live Server, la aplicación debe mostrar los tickets, los filtros, el formulario y la indicación de que los datos fueron cargados desde una API.

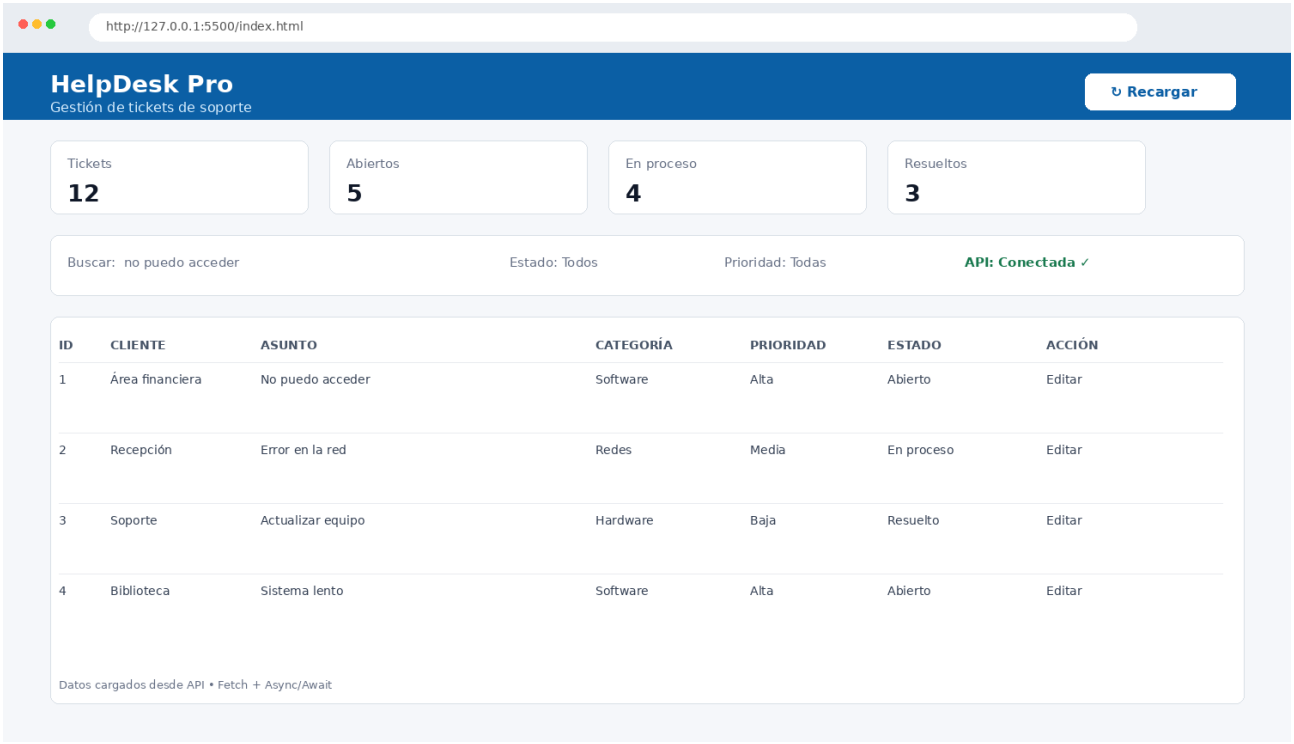


Figura 11. Resultado esperado de la aplicación en el navegador.

17. Actividad práctica — Aplicar todo a la app

El estudiante trabajará exclusivamente sobre la aplicación HelpDesk. La actividad se considera terminada cuando el flujo completo funcione:

1. Abrir la aplicación.
2. Consultar tickets mediante GET.
3. Mostrar 'Cargando...' mientras se espera la respuesta.
4. Transformar los datos.
5. Renderizar la tabla.
6. Buscar tickets.
7. Filtrar por estado.
8. Filtrar por prioridad.
9. Crear un ticket mediante POST.
10. Mostrar un mensaje si ocurre un error.
11. Eliminar un ticket mediante DELETE (reto).

18. Distribución de las 3 horas

Tiempo	Actividad
15 min	Explicación del problema y revisión del Lab. 05
20 min	Estructura HTML de la aplicación
25 min	Creación de api.js y GET
25 min	Transformación de datos
30 min	Renderizado, búsqueda y filtros
25 min	POST desde el formulario
15 min	Manejo de carga y errores
20 min	DELETE y delegación de eventos
15 min	Pruebas y evidencias

19. Pruebas obligatorias

Prueba	Acción	Resultado esperado
1	Abrir la aplicación	Se muestra el estado de carga y luego los tickets.
2	Buscar por cliente	La tabla muestra coincidencias.
3	Filtrar por prioridad Alta	Solo se muestran tickets Alta.
4	Filtrar por estado	Se muestran los tickets del estado elegido.
5	Crear ticket	El formulario envía el registro y la tabla se actualiza.
6	Desconectar/simular error	Aparece mensaje de error.
7	Eliminar	El ticket desaparece de la interfaz.

20. Retos de profundización

8. Implementar PUT para editar un ticket.
9. Agregar botón 'Recargar tickets'.

10. Mostrar la cantidad de tickets cargados.
11. Mostrar fecha y hora de última actualización.
12. Agregar filtro por categoría.
13. Crear un indicador de tickets por estado.
14. Agregar modal para ver el detalle del ticket.
15. Preparar el código para reemplazar JSONPlaceholder por un backend propio.

21. Evidencias de entrega

- Captura de la aplicación del Laboratorio 05 antes de modificarla.
- Captura de la estructura del nuevo proyecto.
- Captura de index.html.
- Captura de api.js.
- Capturas de app.js por bloques.
- Captura de styles.css.
- Captura del resultado en navegador.
- Captura de las pruebas de búsqueda y filtros.
- Captura de creación de un ticket.
- Captura del manejo de error.
- README explicando el funcionamiento.

22. Preguntas de sustentación

16. ¿Qué es una API?
17. ¿Qué hace fetch()?
18. ¿Qué diferencia existe entre GET, POST, PUT y DELETE?
19. ¿Qué función cumple async?
20. ¿Qué función cumple await?
21. ¿Qué representa response.json()?
22. ¿Por qué se creó api.js separado de app.js?
23. ¿Qué hace transformarTicket()?
24. ¿Cómo llega la información de la API hasta la tabla?
25. ¿Por qué se mantiene el array tickets aunque los datos vengan de una API?
26. ¿Qué sucede si la API no responde?
27. ¿Cómo se mantiene la búsqueda después de integrar la API?
28. ¿Cómo funciona la delegación de eventos?
29. ¿Qué cambiaría si se conecta un backend propio?
30. ¿Dónde quedaría almacenada la información de manera permanente en una arquitectura real?

23. Rúbrica de evaluación

Criterio	Porcentaje	Evidencia
Integración con API	20%	GET funcional y datos visibles

Fetch / Async-Await	15%	Código asíncrono correcto
Integración con aplicación anterior	15%	Continuidad del HelpDesk
DOM y renderizado	15%	Tabla dinámica
Búsqueda y filtros	10%	Filtros funcionando
Manejo de errores	10%	Estados de carga y error
POST / CRUD	10%	Registro y reto de eliminación
Sustentación	5%	Explicación del código

24. Proyección — Laboratorio 07

La aplicación queda preparada para el siguiente paso: sustituir la API de práctica por una API propia desarrollada en backend y conectarla a una base de datos.

Laboratorio 04
Arrays + análisis de datos
↓
Laboratorio 05
DOM + eventos + CRUD de interfaz
↓
Laboratorio 06
API + Fetch + Async/Await
↓
Laboratorio 07
Backend + API propia + Base de datos

La meta es que el estudiante entienda que no está realizando aplicaciones aisladas, sino construyendo progresivamente una misma solución empresarial.

25. Nota docente

Los pantallazos incluidos en este documento representan visualmente el código que debe desarrollarse en Visual Studio Code, con estructura, archivos, numeración y presentación de editor. El estudiante debe reproducir y ejecutar el código en su propio entorno para obtener las evidencias de funcionamiento.